

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD PIPELINE DESIGN EXERCISE

Course Code:	CSA1013	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
Student Name:	SURENDAR M G	Reg. No.:	192521374
Date:	20-08-2026	Duration:	60 minutes

CO4 Statement: Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics

Project / Problem Statement

AI-Powered Smart Forest Fire Detection and Environmental Monitoring System

CI/CD Pipeline Design Document

Code of Conduct

I, **SURENDAR M G (Reg. No. 192521374)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Rubrics for Evaluation

The CI/CD Pipeline Design Exercise (25 marks total) is evaluated against the following criteria, reproduced from the assignment's official rubric sheet.

Evaluation Criteria	Excellent	Good	Satisfactory	Needs Improvement
Requirement & Pipeline Analysis (15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow (25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection (20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are appropriate but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration (15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy (15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and traceability to stages	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity in places	Poorly presented, incomplete, or difficult to follow diagram/documentation

Total Maximum Marks: 25 (15% + 25% + 20% + 15% + 15% + 10% of total, weighted as shown above).

Table of Contents

1. Introduction and Purpose
2. CI/CD Needs Analysis
3. Pipeline Architecture and Workflow
4. Tools and Technology Selection
5. Automated Testing and Quality Integration Strategy
6. Deployment Environments and Strategy
7. Security, Quality Gates and Notifications
8. Rollback and Resilience Strategy
9. Sample Pipeline Configuration
10. CI/CD Pipeline Diagram and Workflow Explanation
11. Traceability to Project Requirements
12. Pipeline KPIs and DORA Metrics
13. Conclusion

1. Introduction and Purpose

This document presents the CI/CD (Continuous Integration and Continuous Deployment) Pipeline Design deliverable for the **AI-Powered Smart Forest Fire Detection and Environmental Monitoring System**, prepared in response to the CO4 Assessment Tool 2 instructions. The system automates wildfire detection by fusing IoT ground-sensor telemetry, AI/computer-vision analysis, drone and satellite imagery, and weather/drought data into a continuous monitoring and early-warning platform composed of eight cooperating modules (Section 4 of the project's Software Requirement Specification).

Because the platform's core value depends on a fast, reliable and secure sense-detect-verify-alert chain, manual build-test-release cycles are unacceptably slow and error-prone. This document therefore designs an automated CI/CD pipeline that integrates, tests, quality-checks, secures, packages and deploys every code change through Development, Staging and Production environments, with automated rollback if a release degrades detection latency, dashboard performance, or system availability.

The design follows the instructions specified for this assignment: it analyses the project's CI/CD needs, designs the pipeline workflow and stages, identifies and justifies suitable tools for each stage, defines the automated testing strategy, designs the deployment strategy across environments, incorporates security, quality, notification and rollback mechanisms, and closes with a full pipeline diagram explaining the workflow from code commit to production deployment.

2. CI/CD Needs Analysis

The table below analyses the assigned problem statement and its SRS to identify why each CI/CD capability is required for this specific system, rather than treating CI/CD as a generic best practice.

CI/CD Need	Why This System Requires It
Multiple independently-deployable services	The platform is composed of an IoT ingestion layer, an AI detection/risk-scoring layer, a notification/escalation layer, a reporting layer, and a command dashboard — each must be built, tested and deployed independently without blocking the others.
Frequent, safe integration of contributions	Backend, frontend, ML and QA engineers commit changes continuously; a CI pipeline is needed to integrate, compile and test every change automatically before it reaches a shared branch.
High confidence in AI-detection correctness	Because a missed or false fire-detection alert has real-world safety consequences, the pipeline must run automated unit, integration and model-regression tests before any change to the detection or risk-scoring services is deployed.
Strict non-functional targets	60-second alert generation, 2-second dashboard load, and 99.5% uptime are hard constraints (Section 6, SRS) that must be continuously validated by automated performance checks in the pipeline rather than only at release time.
Security & compliance of sensitive data	Location and personal contact data of field officers and communities is safety-sensitive (Section 11.2, SRS); the pipeline must include automated security and dependency scanning before every deployment.
Zero/near-zero downtime for the alerting path	The alerting and dashboard services must remain available during deployments (NFR-19); the deployment strategy therefore needs a blue-green or canary approach with automated rollback.
Reproducible, containerised environments	Field, staging and production environments must behave identically to avoid environment-specific detection failures; this requires containerisation and infrastructure-as-code.

3. Pipeline Architecture and Workflow

The pipeline is organised into three logical phases — **Continuous Integration (CI)**, embedded **Quality & Security Gates**, and **Continuous Delivery/Deployment (CD)** — executed as a sequential, gated workflow. Every stage must pass before the next stage begins; a failure at any stage stops the pipeline and notifies the responsible team (Section 7).

High-level workflow:

- Code Commit → Source Control & CI Trigger → Build & Compile → Automated Unit Tests
- → Static Code Analysis → Security & Dependency Scan → Package & Push Container Image
- → Deploy to Staging → Automated Integration/E2E Tests → Manual Approval Gate
- → Deploy to Production (Blue-Green) → Continuous Monitoring & Auto-Rollback

The complete stage-by-stage breakdown, with the trigger condition and purpose of each stage, is given below.

#	Stage	Tool(s)	Trigger	Purpose
1	Source Control & Trigger	GitHub (branch protection on main) + GitHub Actions webhook	Push / Pull Request	Every commit or PR automatically triggers the pipeline; branch protection prevents direct pushes to main.
2	Build & Compile	Docker multi-stage builds; Maven (Java services), npm (React dashboard), pip (Python AI service)	On trigger	Produces reproducible, versioned build artefacts / container images for each microservice.
3	Unit & Component Testing	PyTest (AI/risk-scoring service), JUnit 5 (backend services), Jest + React Testing Library (dashboard)	After build	Fails fast on logic errors before expensive integration stages run.
4	Static Code Analysis	SonarQube (code smells, complexity, duplication, coverage gate)	After unit tests	Enforces coding standards and blocks merges below the configured quality gate.
5	Security & Dependency Scan	Trivy (container image scan), OWASP Dependency-Check / Snyk (library CVEs), Gitleaks (secret scanning)	After static analysis	Blocks the pipeline on critical CVEs or leaked credentials before packaging.
6	Package & Push Artifact	Docker + Amazon ECR (private registry)	After security scan passes	Immutable, versioned images are the single deployable unit for every environment.
7	Deploy to Staging	Kubernetes (Amazon EKS) + Helm charts	Automatic on main branch build	Mirrors production topology so integration issues surface before real users are affected.
8	Automated Integration & E2E Tests	Postman/Newman (API contract tests), Selenium & Cypress (dashboard E2E), Locust (load test on alert path)	After staging deploy	Validates the full sense-detect-verify-alert chain end to end, including the 60-second/2-second timing targets.
9	Manual Approval Gate	GitHub Environments protection rule + QA/Lead sign-off	After E2E tests pass	Gives a human checkpoint before changes reach the safety-critical production alerting path.
10	Deploy to Production	ArgoCD (GitOps) + Kubernetes Blue-Green/Canary rollout	On approval	Shifts traffic gradually; the previous version stays warm for instant rollback.
11	Post-Deployment Monitoring	Prometheus (metrics) + Grafana (dashboards) + Alertmanager	Continuous after deploy	Continuously verifies latency/error-rate SLOs; abnormal metrics auto-trigger rollback.

#	Stage	Tool(s)	Trigger	Purpose
1 2	Notifications	Slack + Email (via webhook/SMTP integration)	On every stage transition	Keeps the whole Scrum team and QA immediately informed of build, test, security and deploy outcomes.

3.1 Branching and Release Strategy

A lightweight trunk-based-development strategy is used so the pipeline can integrate changes continuously without long-lived branches drifting out of sync with a fast-evolving detection model.

Branch	Purpose	Pipeline Behaviour
main	Always releasable; protected; only updated via approved Pull Request	Deploys automatically to Staging on every merge
feature/*	Short-lived branch per user story / bug fix (e.g. feature/fr-4.3-stale-flag)	CI runs build, unit tests and static analysis on every push
release/x.y	Cut from main immediately before a Production release for final regression and stakeholder sign-off	Only this branch can trigger the Manual Approval Gate → Production deploy
hotfix/*	Emergency fix branched directly from the last Production tag	Fast-tracked through the same security/quality gates, skipping non-blocking Could-priority checks only

Every merge to main and every Production release is tagged using semantic versioning (e.g. v1.4.0), and the tag is written back into the ArgoCD application history so that any deployed artefact can be traced to an exact commit — satisfying the audit-logging expectations of NFR-8.

4. Tools and Technology Selection

Tools were selected to match the technology stack already implied by the SRS (Python AI service, Java/Node backend services, React dashboard, Kubernetes-friendly containerised deployment) and to keep the toolchain open-source-first and CNCF-standard wherever possible, minimising licensing cost for a pilot government deployment.

Category	Selected Tool(s)	Justification
Source Control	GitHub + branch protection	Industry-standard, integrates natively with GitHub Actions and supports required PR review/approval gates.
CI Orchestration	GitHub Actions	No separate CI server to maintain; free for the pilot's usage volume; first-class Docker/Kubernetes actions available.
Containerisation	Docker (multi-stage builds)	Guarantees the Dev/Staging/Production environment parity required to avoid field-vs-lab detection discrepancies.
Unit Testing	PyTest, JUnit 5, Jest	Matches each service's implementation language; all three integrate with coverage reporting for the SonarQube gate.
Code Quality	SonarQube	Provides a single quality gate (coverage, duplication, complexity, code smells) enforced identically across all services.
Dependency / Container Scanning	Trivy, OWASP Dependency-Check, Gitleaks	Free, actively maintained, and cover the three most common supply-chain risks: vulnerable OS packages, vulnerable libraries, and leaked secrets.
Container Registry	Amazon ECR	Private, IAM-integrated registry that pairs naturally with an EKS-based deployment target.
Orchestration / Deployment	Kubernetes (Amazon EKS) + Helm	Supports independent scaling of the eight modules and the multi-zone scale-out roadmap (Section 3.2, SRS) without redesign.
Integration / E2E Testing	Postman/Newman, Selenium, Cypress, Locust	Covers API-contract, browser-UI and load-testing needs so the 60-second alert and 2-second dashboard SLOs are continuously verified.
GitOps Continuous Delivery	ArgoCD	Declarative, auditable deployment history — important given the platform's audit-logging and traceability requirements (NFR-8).
Monitoring & Alerting	Prometheus + Grafana + Alertmanager	Open-source, Kubernetes-native, and directly supports the automated-rollback health checks in Section 8.
Notifications / ChatOps	Slack + Email (SMTP/webhook)	Matches the multi-channel alerting philosophy already used in the product itself (FR-7.1), applied here to the release process.

5. Automated Testing and Quality Integration Strategy

Testing is layered so that cheaper, faster tests run first and catch the majority of defects before expensive environment-dependent tests run. This mirrors the classic test pyramid, adapted to this system's safety-critical detection path.

5.1 Unit & Component Testing (fastest feedback)

Runs on every commit/PR. Covers business logic in isolation: AI confidence-threshold decisions, risk-score categorisation, sensor anomaly-flag rules, and RBAC role checks. A minimum 80% line-coverage gate is enforced by SonarQube before merge is allowed.

5.2 Static Code Analysis & Code-Quality Gates

SonarQube evaluates maintainability, duplication and cyclomatic complexity on every build; a build is marked failed if the quality gate is not met, preventing quality debt from compounding across 32 functional and 22 non-functional requirements.

5.3 Security & Dependency Testing

SAST, SCA, container-image scanning and secret scanning run automatically before packaging (see Section 7 for the full security-gate table), directly enforcing NFR-5, NFR-6 and NFR-7 from the SRS as pipeline gates rather than manual release checklists.

5.4 Integration & End-to-End Testing

Executed against the Staging environment after every successful deploy: Postman/Newman verifies API contracts between the ingestion, AI-detection, risk-scoring and notification services; Selenium/Cypress drives the React Command Dashboard through login, alert-verification and filtering flows; Locust load-tests the alert path to confirm the 60-second (FR-3.1/FR-7.1) and 2-second (FR-8.1) timing targets hold under simulated peak sensor volume.

5.5 Model-Regression Testing (AI/CV service)

Because the fire-detection model is periodically retrained on newly verified/false-positive feedback (FR-3.3), the pipeline runs a fixed regression image-set through the candidate model and blocks promotion if precision/recall on that set regresses beyond an agreed tolerance versus the currently deployed model.

6. Deployment Environments and Strategy

Three environments are used, each with a distinct purpose, infrastructure footprint and access-control policy, so that a change is proven twice (once in Staging) before it can affect a live forest zone.

Environment	Purpose	Infrastructure	Data	Access	Deployment Trigger
Development	Rapid local/dev-cluster iteration for feature branches	Docker Compose on developer machines + a shared dev namespace in EKS	Synthetic/seeded sensor & alert data	Any engineer with repo write access	Automatic on every feature-branch push
Staging	Full integration & E2E verification mirroring production topology	Dedicated EKS namespace, same Helm charts as production, scaled down	Anonymised copy of historical sensor/incident data	Backend, QA and DevOps engineers	Automatic on every merge to main after CI passes
Production	Live monitoring, detection and alerting for real forest zones	Multi-AZ EKS cluster, autoscaled, blue-green traffic routing	Live IoT/AI/alert data (access-controlled, audit-logged per NFR-6/NFR-8)	System Administrators + release approvers only	Manual approval gate, then automated blue-green rollout

6.1 Deployment Strategy

Production deployments use a **Blue-Green rollout** managed by ArgoCD: the new (“Green”) version is deployed alongside the running (“Blue”) version inside the same EKS cluster, health-checked, and only then given live traffic via a Kubernetes Service/Ingress switch. High-risk changes — particularly to the AI detection model — use a **Canary rollout** instead, exposing the new version to a small percentage of zone traffic before a full promotion (see Section 8).

7. Security, Quality Gates and Notifications

7.1 Security Checks Embedded in the Pipeline

Security Check	Tool	Pipeline Stage	Why It Matters
Static Application Security Testing (SAST)	SonarQube security rules	Static Code Analysis stage	Flags injection, hard-coded secrets and insecure coding patterns pre-merge.
Software Composition Analysis (SCA)	OWASP Dependency-Check / Snyk	Security & Dependency Scan stage	Blocks known-CVE libraries (e.g., outdated MQTT/broker client) from shipping.
Container Image Scanning	Trivy	Security & Dependency Scan stage	Scans OS packages and base-image layers of every Docker image before push.
Secret Scanning	Gitleaks (pre-commit + CI)	Source commit & CI trigger	Prevents API keys, DB credentials or SMS-gateway tokens from ever reaching the repo.
Dynamic Application Security Testing (DAST)	OWASP ZAP baseline scan	Post-staging-deploy	Probes the running staging environment for runtime vulnerabilities (XSS, auth bypass).
RBAC / Access-Control Regression Tests	Custom Postman/Newman suite	Automated Integration & E2E stage	Re-verifies NFR-6 (API-layer RBAC) on every build so a regression cannot silently reopen an authorisation gap.
Infrastructure-as-Code Scanning	Checkov / tfsec on Terraform	Pre-deploy (IaC pipeline)	Catches misconfigured security groups, public S3 buckets, or over-privileged IAM roles before they're provisioned.

7.2 Notifications

Every meaningful pipeline event notifies the right audience through the channel best suited to its urgency:

- **Build/Test failure:** Slack #ci-alerts channel + email to the commit author, including the failing stage and log excerpt link.
- **Security scan failure (critical CVE / secret leak):** Immediate Slack DM to the DevOps/Security lead and pipeline hard-stop; deployment cannot proceed.
- **Staging deployment success:** Slack #qa-staging channel notified so QA can begin E2E verification immediately.
- **Manual approval pending:** Slack + email reminder to the designated release approver(s) with a direct approval link.
- **Production deployment success/failure:** Slack #releases channel and email to the Product Owner and Forest Fire Management Authority stakeholders (per Section 1.3 of the SRS).
- **Automated rollback triggered:** High-priority Slack/PagerDuty alert to the on-call DevOps engineer, since this indicates a live production issue.

8. Rollback and Resilience Strategy

Because the alerting and dashboard services must remain available during deployment (NFR-19) and a failed release on the safety-critical detection path is unacceptable, rollback is treated as a first-class, automated pipeline capability rather than a manual emergency procedure.

- **Deployment strategy:** Blue-Green rollout via ArgoCD/Kubernetes — the new version (“Green”) is deployed alongside the running version (“Blue”) and traffic is shifted only after health checks pass, so the previous version stays warm and ready.
- **Automated rollback triggers:** HTTP 5xx error-rate above 2% for 5 minutes, fire-detection alert-generation latency exceeding the 60-second SLO, dashboard load time exceeding 2 seconds, or a failing Kubernetes readiness/liveness probe.
- **Rollback mechanism:** `kubectl rollout undo` / ArgoCD automated sync-rollback reverts traffic to the last known-good “Blue” version within seconds, with no manual intervention required for the alerting path.
- **Manual rollback:** A one-click “Revert Release” action in the ArgoCD UI / Slack ChatOps command is available to the Release Manager for issues not caught by automated health checks.
- **Canary option for high-risk changes:** Changes to the AI fire-detection model are rolled out to 10% of zone traffic first, monitored for detection-accuracy drift, and only promoted to 100% after a defined observation window.
- **Database migration safety:** All schema migrations are written to be backward-compatible for one release cycle so a code rollback never leaves the database in an incompatible state.

9. Sample Pipeline Configuration

The following abridged GitHub Actions workflow illustrates how the stages designed in Section 3 are expressed as executable pipeline configuration.

```
name: forest-fire-platform-ci-cd

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build services (Docker multi-stage)
        run: docker compose -f build/docker-compose.ci.yml build

      - name: Run unit tests (AI service)
        run: pytest ai-service/tests --cov=ai_service --cov-report=xml

      - name: Run unit tests (backend services)
        run: mvn -B test -f backend/pom.xml

      - name: Run unit tests (dashboard)
        run: npm ci --prefix dashboard && npm test --prefix dashboard -- --ci

      - name: Static code analysis
        uses: sonarsource/sonarqube-scan-action@v2
        with:
          args: -Dsonar.qualitygate.wait=true

      - name: Dependency & secret scan
        run: |
          trivy fs --exit-code 1 --severity CRITICAL,HIGH .
          gitleaks detect --no-git -v

      - name: Build & push container images
        run: |
          docker build -t $ECR_REGISTRY/ff-ai-service:${{ github.sha }} ./ai-service
          docker push $ECR_REGISTRY/ff-ai-service:${{ github.sha }}

  deploy-staging:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Helm upgrade (staging namespace)
        run: helm upgrade --install ff-platform ./charts -n staging -f values-staging.yaml

      - name: Run integration & E2E tests
        run: newman run tests/api-collection.json && npx cypress run

  approval-gate:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production-approval
    steps:
      - run: echo "Awaiting manual QA/Lead approval before production deploy"

  deploy-production:
    needs: approval-gate
    runs-on: ubuntu-latest
    steps:
      - name: Blue-Green rollout via ArgoCD
        run: argocd app sync ff-platform-prod --strategy blue-green

      - name: Post-deploy health check & auto-rollback
        run: ./scripts/verify_slo.sh || argocd app rollback ff-platform-prod
```

10. CI/CD Pipeline Diagram and Workflow Explanation

The diagram below shows the complete pipeline from code commit to production deployment, colour-coded by phase: blue for Continuous Integration, orange for Quality & Security Gates, and green for Continuous Delivery/Deployment.

CONTINUOUS INTEGRATION

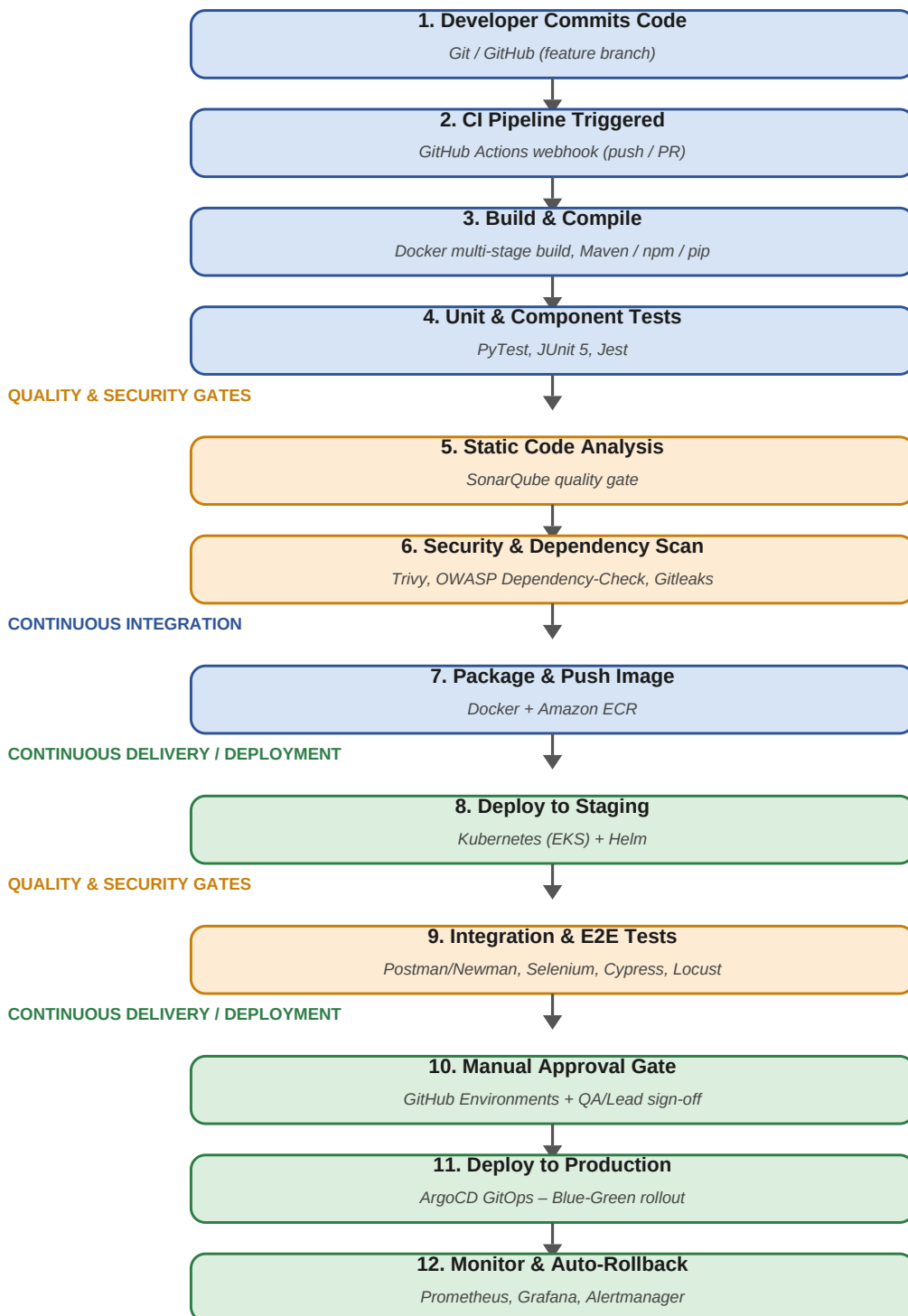


Figure 10.1 – End-to-end CI/CD pipeline for the AI-Powered Smart Forest Fire Detection and Environmental Monitoring System.

10.1 Workflow Walkthrough

1. A developer commits code to a feature branch and opens a Pull Request against main.
2. GitHub Actions automatically triggers the CI pipeline on the push/PR event.
3. Each affected service (AI service, backend APIs, dashboard) is built inside a Docker multi-stage build.
4. Unit and component tests run per service; the build fails fast if any test fails.
5. SonarQube performs static code analysis against the configured quality gate.
6. Trivy, OWASP Dependency-Check and Gitleaks scan for vulnerable dependencies, vulnerable OS packages and leaked secrets.
7. On success, versioned container images are built and pushed to Amazon ECR.
8. ArgoCD/Helm automatically deploys the new images to the Staging namespace in EKS.
9. Postman/Newman, Selenium, Cypress and Locust run automated integration, UI and load tests against Staging, validating the 60-second alert and 2-second dashboard targets.
10. A manual approval gate requires QA/Lead sign-off before the change can reach Production, given the safety-critical nature of the alerting path.
11. On approval, ArgoCD performs a Blue-Green (or Canary, for AI-model changes) rollout to Production.
12. Prometheus/Grafana continuously monitor latency, error rate and availability post-deploy; Alertmanager triggers an automatic rollback if any SLO is breached.
13. Slack and email notifications are sent at every major transition, keeping the Scrum team, QA, and business stakeholders informed in real time.

11. Traceability to Project Requirements

The table below confirms that every relevant non-functional requirement from the project's SRS is directly enforced by a specific stage of this CI/CD pipeline, so pipeline design is not generic but demonstrably tailored to this system.

Requirement	Concern	Enforced By
NFR-1 / FR-3.1 / FR-7.1	60-second alert-generation / dispatch target	Stage 9 – Locust load test asserts p95 latency ≤ 60 s
NFR-2 / FR-8.1	2-second dashboard load target	Stage 9 – Cypress + synthetic monitoring assert dashboard TTI ≤ 2 s
NFR-5, NFR-7	Password hashing, TLS enforcement	Stage 5/6 – SonarQube security rules + DAST scan on Staging
NFR-6	API-layer RBAC enforcement	Stage 9 – dedicated Postman/Newman RBAC regression suite
NFR-9, NFR-19	99.5% uptime, zero-downtime deploys	Stage 11 – Blue-Green rollout with automated health-check rollback
NFR-8	Immutable, attributable audit logging	Stage 11 – ArgoCD GitOps history provides a full, auditable deployment trail
FR-3.3	Model retraining on verified feedback	Stage 4/9 – Model-regression test gate before promoting a retrained model
Section 3.2 (scale-out)	Single-zone pilot to multi-zone scale-out	Stage 8 – Kubernetes/Helm design keeps services independently scalable

12. Pipeline KPIs and DORA Metrics

To confirm the pipeline is actually improving delivery performance — not just automating existing practice — the team tracks the four DORA (DevOps Research and Assessment) metrics alongside two project-specific targets tied directly to the wildfire-detection mission.

Metric	Definition	Target / Use	Measured Via
Deployment Frequency	How often Production is successfully deployed	Target: multiple times per week during active sprints	ArgoCD deployment history
Lead Time for Changes	Time from commit to running in Production	Target: under 1 day for a Should/Must-priority fix	GitHub Actions + ArgoCD timestamps
Change Failure Rate	% of deployments that trigger a rollback or hotfix	Target: below 15%	Alertmanager rollback events / total deployments
Mean Time to Restore (MTTR)	Time from a failed deployment to automatic/manual rollback completion	Target: under 5 minutes (matches NFR-19 availability goal)	Prometheus/Grafana incident timestamps
Detection-Path SLO Compliance	% of releases that keep the 60-second alert / 2-second dashboard targets within budget	Target: 100% — a release breaching this is auto-rolled back	Locust/Cypress synthetic checks in Stage 9 and post-deploy monitoring
Security Gate Pass Rate	% of builds that pass SAST/SCA/container scans on the first attempt	Tracked to identify recurring dependency-hygiene issues	SonarQube + Trivy dashboards

13. Conclusion

This CI/CD Pipeline Design document has translated the AI-Powered Smart Forest Fire Detection and Environmental Monitoring System problem statement and its Software Requirement Specification into a complete, justified pipeline covering source control, build, automated testing, static and dynamic security analysis, containerised packaging, a three-environment deployment strategy, notifications, and an automated Blue-Green/Canary rollback mechanism.

Every stage was chosen and justified against the specific needs of a safety-critical wildfire-detection platform rather than generic DevOps practice, and every non-functional requirement from the SRS was traced to the pipeline stage that continuously enforces it. This gives the development team, QA, and evaluators confidence that every release can be integrated, verified, secured and deployed rapidly without compromising the reliability of the sense-detect-verify-alert chain the project depends on.

End of CI/CD Pipeline Design Document.

Appendix A – Glossary of Terms

Term	Definition
CI	Continuous Integration – automatically building and testing every code change.
CD	Continuous Delivery/Deployment – automatically releasing verified changes toward Production.
SAST	Static Application Security Testing – scanning source code for security flaws without executing it.
SCA	Software Composition Analysis – scanning third-party dependencies for known vulnerabilities.
DAST	Dynamic Application Security Testing – scanning a running application for runtime vulnerabilities.
GitOps	A delivery model where the desired system state is declared in Git and continuously reconciled by a controller (e.g., ArgoCD).
Blue-Green Deployment	A release strategy that runs the new and old versions side by side and switches traffic only after health checks pass.
Canary Deployment	A release strategy that exposes a new version to a small percentage of traffic before a full rollout.
SLO	Service Level Objective – a measurable target for a system quality attribute, e.g. 60-second alert latency.
DORA Metrics	Four key software-delivery performance metrics defined by the DevOps Research and Assessment team: Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Mean Time to Restore.
RBAC	Role-Based Access Control – restricting system actions based on a user's assigned role.
IaC	Infrastructure as Code – defining and provisioning infrastructure through versioned configuration files (e.g., Terraform).